TÉCNICAS DE INTELIGÊNCIA ARTIFICIAL (PL)

LICENCIATURA EM ENGENHARIA E GESTÃO DE SISTEMAS DE INFORMAÇÃO

JÚLIO DUARTE

julio.duarte@algoritmi.uminho.pt

LISTAS NO PROLOG

- Lista é uma das estruturas simples em Prolog, muito comum em programação não numérica.
- É uma sequência ordenada de elementos.
- Uma lista pode ter qualquer comprimento.
- Por exemplo uma lista de elementos tais como ana, tênis, pedro pode ser escrita em Prolog como:

  [ana, tênis, pedro]

LISTAS NO PROLOG

- A utilização de [ ] é uma melhoria da notação, pois internamente listas são representadas como árvores, assim como todos objetos estruturados em Prolog.

- Para entender a representação no Prolog de listas, é necessário considerar dois casos:
  - A lista é vazia, escrita como [ ].
  - Uma lista (não vazia) consiste:
    - no primeiro item, chamado cabeça (head) da lista.
    - na parte restante da lista, chamada cauda (tail).

LISTAS NO PROLOG

- No exemplo [ana, tênis, pedro]:
  - ana é a Cabeça da lista.
  - [tênis, pedro] é a Cauda da lista.

- A cabeça de uma lista pode ser qualquer objeto (inclusive uma lista); a cauda tem que ser uma lista.

- A Cabeça e a Cauda são combinadas numa estrutura pelo functor especial .

  .(Cabeça, Cauda)

- Como a Cauda é uma lista, ela é vazia ou ela tem sua própria cabeça e sua cauda.

LISTAS NO PROLOG

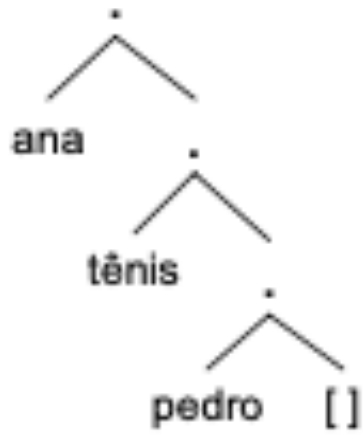
- Assim, para representar listas de qualquer comprimento, nenhum princípio adicional é necessário.

- O exemplo [ana, tênis, pedro] é representando como o termo:

.(ana, .(tênis, .(pedro, [ ])))

Podem ser usadas ambas as notações.

```
?- Lista1 = [a,b,c],
   Lista2 = .(a,.(b,.(c,[]))).
Lista1 = [a, b, c]
Lista2 = [a, b, c]
```



LISTAS NO PROLOG

- Em geral, é comum tratar a cauda como um objeto simples.

- Por exemplo, L = [a,b,c] pode ser escrito como:
  - Cauda = [b,c]
  - L = .(a,Cauda)

- Para expressar isso, Prolog usa uma notação alternativa, a barra vertical, que separa a cabeça da cauda:
  - L = [a | Cauda]

- A notação é geral por permitir qualquer número de elementos seja seguido por | e o restante da lista:

  [a,b,c] = [a | [b,c]] = [a,b | [c]] = [a,b,c | [ ]]

UNIFICAÇÃO EM LISTAS

Lista1	Lista2	Lista1 = Lista2
[mesa]	[X|Y]	X=mesa Y=[ ]
[a,b,c,d]	[X,Y|Z]	X=a Y=b Z=[c,d]
[[ana,Y]|Z]	[[X,foi],[ao,cinema]]	X=ana Y=foi Z=[[ao,cinema]]
[ano,bissexto]	[X,Y|Z]	X=ano Y=bissexto Z=[ ]
[ano,bissexto]	[X,Y,Z]	Não unifica

RECURSÃO EM LISTAS

- Frequentemente, é necessário realizar operações em listas, por exemplo, procurar um elemento que faça parte de determinada lista.
- Para isso, a recursão é o recurso mais usado.
- Para verificar se um nome está na lista, é preciso verificar se ele está na cabeça ou se está na cauda da lista.
- Se for atingido o final da lista, o nome não está na lista.

LISTAS NO PROLOG

- Inicialmente, é necessário definir o nome do predicado que verifica se um elemento X pertence ou não a uma lista L, por exemplo, pertence(X, L).

- A primeira condição especifica que um elemento X pertence à lista se ele está na cabeça dela. Isto é indicado como:

  pertence(X,[X|Z]).

- A segunda condição especifica que um elemento X pertence à lista se ele pertencer à sua cauda. Isto pode ser indicado como:

  pertence(X, [W|Z]) :- pertence(X,Z).

LISTAS NO PROLOG

- Sempre que um programa recursivo é definido, deve-se procurar pelas condições limites (ou condições de paragem) e pelo caso(s) recursivo(s):

```
pertence(Elemento,[Elemento|Cauda]).
pertence(Elemento,[Cabeca|Cauda]) :-
          pertence(Elemento,Cauda).
```

- Após a definição do programa, é possível interrogá-lo. Por exemplo,

```
?- pertence(a,[a,b,c]).
yes
```

```
?- pertence(d,[a,b,c]).
no
```

```
?- pertence(X,[a,b,c]).
X = a ;
X = b ;
X = c ;
no
```